
Assignment 3

TinyML Cosine Wave Predictor

Group Size: same as project size

1. Problem Statement

TinyML is the field of running machine learning models directly on small, resource-constrained microcontrollers. Instead of streaming raw sensor data to the cloud for inference, the model lives on the device itself, enabling low-latency predictions, offline operation, lower bandwidth costs, and improved privacy. Typical TinyML targets have tens of kilobytes of RAM, no operating system, and clock speeds in the hundreds of megahertz at most.

In this assignment, you will build the canonical **"Hello World" of TinyML**: train a small neural network to approximate a trigonometric function, then deploy it on an **ESP32** development board. **To distinguish this assignment from the well known sine wave demo**, you will instead approximate the **cosine function**, $y = \cos(x)$, for $x \in [0, 2\pi]$. The trained model is converted to TensorFlow Lite Micro format, fully quantized to 8-bit integers, and flashed to the ESP32 via the Arduino IDE. The board then runs inference live and streams predicted vs. actual values to the Serial Plotter.

The goal is not high accuracy on a hard problem, $\cos(x)$ is trivial in floating point, but to walk the complete **edge AI pipeline**: data generation, training, quantization, conversion to a C byte array, on-device inference, and visualization. The lessons learned here transfer directly to real TinyML applications such as keyword spotting, gesture recognition, and anomaly detection on sensor data.

Note: No additional hardware is required beyond the ESP32 board and a USB cable. All inputs are generated internally on the device; all outputs are observed through the Arduino Serial Monitor / Serial Plotter.

2. Pipeline Overview

Stage	Details
Data	Generate 1000 samples: $x \in [0, 2\pi]$, $y = \cos(x) + \text{Gaussian noise } (\sigma \approx 0.1)$. Split into train / validation / test (e.g., 60 / 20 / 20).
Model	Small fully connected network: Dense(16, ReLU) $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(1, linear). Single scalar input, single scalar output. ~321 trainable parameters.
Optimization	Full integer (int8) post training quantization using a representative dataset. Inputs and outputs are also int8. Target model size: a few kilobytes.
Deployment	Convert the .tflite file to a C header (model.h) with xxd, include it in an Arduino sketch, run inference using the TensorFlow Lite Micro / TensorFlowLite_ESP32 runtime.
Output	ESP32 sweeps x from 0 to 2π , prints "predicted,actual" over Serial. The Arduino IDE Serial Plotter draws both curves so the approximation can be visually compared to the ground truth cosine wave.

3. Technical Requirements

3.1 Data Generation & Model Training (30 marks)

1. Implement a Python script (or Google Colab) **train_cosine.py** using **TensorFlow 2.x / Keras**.
2. Generate **1000 samples** of $x \in [0, 2\pi]$ (uniform random, fixed seed for reproducibility) and compute $y = \cos(x) + \varepsilon$ where $\varepsilon \sim N(0, 0.1)$. Shuffle and split into 60% training / 20% validation / 20% test.
3. Define the model exactly as: *Dense(16, activation='relu') → Dense(16, activation='relu') → Dense(1)*. Use the Adam optimizer and Mean Squared Error loss.
4. Train for at least 200 epochs (batch size 32 or 64). Plot training and validation loss curves and include them in the report.
5. Evaluate on the held out test set and report MSE and MAE. The float model should reach a test MAE of roughly 0.05 or better.
6. Plot the float model prediction against the ground truth cosine on a dense grid of x values; include the plot in the report.

3.2 Model Optimization Full Integer Quantization (30 marks)

1. Convert the trained Keras model to a TensorFlow Lite model using *tf.lite.TFLiteConverter.from_keras_model*.
2. Enable **full integer (int8) post-training quantization**. You must:
 - Set `converter.optimizations = [tf.lite.Optimize.DEFAULT]`.
 - Provide a `representative_dataset` generator that yields ~100 samples from the training distribution.
 - Set `converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]`.
 - Force `inference_input_type = tf.int8` and `inference_output_type = tf.int8` so the model has no float ops at runtime.
3. Save the quantized model to *cosine_int8.tflite* and verify its size in bytes. Report the size and the size reduction vs. the un quantized float model.
4. Run the quantized model in Python using the TFLite *Interpreter* on the same test set used in 3.1. Report the quantized MAE and overlay the int8 prediction curve on top of the float prediction and ground truth (three curves on one plot).
5. Convert *cosine_int8.tflite* to a C byte array using `xxd -i cosine_int8.tflite > model.h`. Rename the array to `g_model[]` and mark it `alignas(8) const unsigned char` so it is placed in flash and properly aligned for the TFLite Micro runtime.

3.3 ESP32 Deployment & Visualization (40 marks)

1. Install the **ESP32 board package** in the Arduino IDE (Boards Manager → *esp32 by Espressif Systems*) and install a TensorFlow Lite Micro library that supports ESP32 for example *Chirale_TensorFlowLite*, *TensorFlowLite_ESP32 (tanakamasayuki)*, or the official *esp_tflite_micro* component when using PlatformIO / ESP IDF.
2. Create an Arduino sketch **cosine_predictor.ino** that includes *model.h* and sets up the TFLite Micro interpreter with:
 - A *tflite::MicroMutableOpResolver* registering only the ops the model actually uses (FullyConnected, Relu). Smaller resolvers save flash.
 - A static tensor arena of approximately 2–4 KB (start with 4 KB; tune downward and report the smallest size that still runs).
 - An interpreter created from the model and arena, with *AllocateTensors()* called once in *setup()*.
3. In *loop()*, sweep x from 0 to 2π in fixed steps (e.g., 80 steps per period). For each step:
 - Quantize the float x into int8 using the input tensor's quantization scale and *zero_point*.
 - Run interpreter *>Invoke()* and read the output tensor.
 - Dequantize the int8 output back into float.
 - Compute the ground truth value with the C math library *cos(x)*.
 - *Serial.print* the two values separated by a comma and terminated by a newline (e.g., "0.81,0.84").
4. Open the **Serial Plotter** (Tools → Serial Plotter) at 115200 baud. Both curves *predicted* and *actual* must be visible simultaneously and visually track each other through at least two full periods. Capture a screenshot for the report.
5. Print at startup (once, in *setup()*): the model arena bytes used, the input/output quantization parameters (*scale*, *zero_point*), and the size of *g_model* in bytes. Include these lines in the report.

4. Deliverables

4.1 Technical Report (PDF)

A well-structured report containing:

1. **Title page** with name(s), student ID(s), course name, and date.
2. Code snippets, Screenshots of your code, and output.

4.2 Source Code (ZIP Archive)

A ZIP file containing:

- `train_cosine.py` (or `train_cosine.ipynb`) data generation, training, evaluation, and TFLite conversion.
- `cosine_float.tflite` and `cosine_int8.tflite` both TFLite files.
- `model.h` the C byte array header generated with `xxd`.
- `cosine_predictor.ino` in the Arduino sketch.
- `plots/` saved PNGs of all figures used in the report.
- `README.md` setup instructions: Python version, TensorFlow version, Arduino IDE setup, board package version, TFLite Micro library name and version, USB port notes.
- `requirements.txt` pinned Python dependencies (tensorflow, numpy, matplotlib).

4.3 Demo Video

A 3–7 minute screen recording demonstrating: (a) running the training script and showing the loss curve and final MAE, (b) the quantization step and the resulting file size, (c) flashing the ESP32 from the Arduino IDE, (d) the Serial Monitor startup output, and (e) the Serial Plotter showing the predicted and actual cosine curves moving together in real time.

5. Grading Rubric

Component	Marks	Key Criteria
Data Generation & Model Training	30	Correct dataset & split, exact architecture, training stability, loss curves, float model accuracy
Quantization & C Conversion	30	Full int8 quantization (incl. I/O), representative dataset, accuracy retained, valid model.h generated with xxd
ESP32 Deployment & Visualization	40	Sketch compiles & runs, correct quant/dequant, predicted vs actual curves visible in Serial Plotter, startup diagnostics

6. Recommended Tools & Resources

6.1 Required Tools

Tool	Purpose	Link / Install
Python 3.10+	Training, evaluation, TFLite conversion	python.org
TensorFlow 2.x	Keras model, TFLite converter, quantization	<code>pip install tensorflow</code>
NumPy + Matplotlib	Data generation, plotting	<code>pip install numpy matplotlib</code>
Arduino IDE 2.x	Compile and flash the ESP32 sketch	arduino.cc/en/software
ESP32 board package	Toolchain & core for ESP32 in Arduino IDE	Boards Manager → "esp32" by Espressif Systems
TFLite Micro Arduino lib	On device inference runtime for ESP32	Chirale_TensorFlowLite OR TensorFlowLite_ESP32 (Library Manager)
xxd	Convert .tflite → C header byte array	Pre installed on Linux/macOS; on Windows use Git Bash or WSL
ESP32 board	Any ESP32 DevKit (DevKit V1, WROOM 32, S3, etc.)	Plus a USB cable (data, not power only)

6.2 Useful References

- TinyML book companion code (sine wave): github.com/tensorflow/tflite-micro/tree/main/tensorflow/lite/micro/examples/hello_world
- TensorFlow Lite post training quantization guide: www.tensorflow.org/lite/performance/post_training_quantization
- TensorFlow Lite Micro overview: www.tensorflow.org/lite/microcontrollers
- ESP32 Arduino core docs: docs.espressif.com/projects/arduino_esp32/en/latest/
- Chirale_TensorFlowLite library: github.com/spaziochirale/Chirale_TensorFlowLite

7. Plagiarism Policy

WARNING: PLAGIARISM AND CHEATING

Any form of plagiarism or copying will result in ZERO marks for the entire project for ALL involved team members.

7.2 How Plagiarism Will Be Detected

1. All submitted Python and Arduino code will be compared against every other team's submission using automated code similarity tools.
2. The exact byte content of model.h, the .tflite file hashes, and architectural fingerprints (layer shapes, training seeds) will be examined.
3. Each team may be asked during the demo to explain any part of their code, model design choices, or quantization settings. Inability to explain your own work is grounds for investigation.

8. Deliverable Discussion / Response Policy

- If a student is asked a question during evaluation and does not answer, 5 points will be deducted.
- If the student fails to answer a second question, an additional 5 points will be deducted.
- If the student fails to answer a third question, the student will receive 0 for the Individual Understanding mark.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark.

9. Important Rules

1. If a late submission is permitted, a 25% penalty will be incurred for each day after the deadline. After two overdue days, no submissions will be accepted.
2. Each deliverable must be uploaded to the assigned Google Classroom assignment and pushed to the repository before the submission deadline.
3. Teams are encouraged to be from the same lab. Team members can be from different labs, but it will be your full responsibility to attend the evaluation session. Any student who misses the scheduled evaluation session will automatically receive ZERO.
4. The deliverable version uploaded to Google Classroom is the one that will be evaluated. If it is a team project, the team representative can upload on behalf of other members. The GitHub version will serve only as a reference to track progress.
5. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. Failing to maintain an updated GitHub repository may result in a penalty of up to 20% for the deliverable under evaluation.
6. **Documentation Requirements:** Each deliverable must include clear and complete documentation as outlined in the tasks.
7. Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable (code, math, board configuration, quantization choices). Any student unable to explain their work may lose marks individually, even if the deliverable was submitted.
8. Cheating may result in a negative grade (up to -10 Course Grades).
9. **Academic Integrity Reminder:** Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct. Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. All submissions are subject to plagiarism and AI detection checks.